



Tema 5

🕒 Tipo	Nota Apuntes
📎 Archivos y multimedia	T5 - Principios SOLID - v6.pdf T5B - Principios RCC-ASS - v4.pdf Inversion_v2.pdf
📄 Bloques de notas	📄 Bloque 2

1. Introducción

Son principios generales de diseño, hoy considerados "ágiles", que reformulan los conceptos clásicos de Cohesión y Acoplamiento en el Diseño Orientado a Objetos (DOO), acuñados por Robert Martin C.

2. S - Principio de Responsabilidad Única (SRP)

Robert define este principio como "Toda clase debería tener un único motivo para cambiar". Es decir, si una clase tiene más de un motivo para cambiar, significa que tiene más de una responsabilidad y debería dividirse en varias clases. Siendo responsabilidad un "encargo asignado a un único actor respecto a una tarea única de negocio".

3. O - Principio Abierto-Cerrado (OCP)

Las entidades de software deben estar abiertas para su extensión pero cerradas para su modificación. Esto se logra mediante la abstracción y el polimorfismo. Por ejemplo, imaginemos una aplicación que simule un tablero de ajedrez.

El tablero tendría un método que llamase a una pieza para moverse. Dicho método tendría que comprobar qué tipo de pieza es para saber cómo moverla. Esto es una implementación que rompe el principio O/C, ya que si quisieramos

añadir una nueva pieza, tendríamos que modificar las funciones del tablero para aceptar este nuevo movimiento.

En lugar de eso, podríamos crear una clase genérica `Pieza` que contenga el método `mover()`. Todas las piezas que queramos añadir serán hijas de esta clase y realizarán su propia implementación del método. De esta forma, el tablero solo tendrá que llamar a la función de la pieza, sin preocuparse por cómo se implementa.

4. L - Principio de Sustitución de Liskov (LSP)

Las subclases deben ser sustituibles por sus clases base. Cualquier propiedad de los objetos supertipo debe cumplirse también en los objetos subtipo. Esto significa también, por consecuencia, que si un método o clase está esperando un objeto de cierta clase, se le puede pasar un hijo suyo sin romper nada. Este principio es complicado romperlo sin querer, puesto que actualmente los tipos suelen ir ligados a las clases.

5. I - Principio de Separación de la Interfaz (ISP)

Los clientes no deben ser forzados a depender de interfaces que no utilizan. Ya que si una interfaz no está cohesionada, los cambios en métodos que un cliente no usa pueden obligarle a recompilar o romperse, generando un acoplamiento innecesario. Por esto, es mejor tener muchas interfaces específicas para cada tipo de cliente que una sola interfaz de propósito general.

6. D - Principio de Inversión de la Dependencia (DIP)

Los módulos de alto nivel no deben depender de los de bajo nivel; ambos deben depender de abstracciones. Esto ayuda a prevenir dependencias de módulos volátiles, ya que las abstracciones cambian con menos frecuencia que las implementaciones concretas.

El OCP establece el objetivo (diseño flexible), mientras que el DIP establece el mecanismo (dependen de abstracciones). Una de las formas de invertir las dependencias sería inyectarlas, aunque hay otras.

6.1 Inyección de Dependencias (DI)

La clave es el rol del Inyector (o Assembler), quien crea las instancias necesarias y las "inyecta" en el cliente. Se distingue de patrones como *Abstract Factory* principalmente por el orden de las operaciones. Tipos de Inyección (Muy importante para Test):

1. Type-1 (Interface Injection): El cliente implementa una interfaz específica que define el método de inyección.
2. Type-2 (Setter Injection): La dependencia se asigna a través de un método *setter* (ej. `setFinder`).
3. Type-3 (Constructor Injection): La dependencia se pasa como parámetro en el momento de crear el objeto mediante su constructor.

7. SOLID y Patrones de Diseño

Los patrones de diseño son realmente, estructuras repetitivas que surgen al seguir estos principios; soluciones probadas a problemas comunes de diseño. Las causas comunes de rediseño que los patrones evitan son crear objetos indicando la clase exacta, dependencia de operaciones o plataformas específicas, acoplamiento fuerte o extender funcionalidad solo mediante subclases.

8. Diseño de Alto Nivel: Paquetes y Componentes

Las clases, aunque útiles como abstracción, no bastan para organizar diseños grandes.

Organizaciones con una granularidad superior a las clases:

- Paquetes: Organización a nivel de código (tiempo de diseño/compilación).
- Componentes: Organización a nivel de distribución (tiempo de ejecución).

- UML: Resume ambos conceptos bajo el término paquetes.

Objetivo: Definir criterios técnicos para particionar clases en paquetes y gestionar sus interrelaciones.

9. Los Principios RCC + ASS (3C/3A)

El autor es Robert C. Martin ("Uncle Bob"), el mismo autor de los 5 SOLID clásicos. Estos principios son fundamentales en aplicaciones grandes donde un pequeño cambio puede propagarse por múltiples dominios y equipos. Se dividen en 3 principios de Cohesión (RCC) y 3 de Acoplamiento (ASS).

9.1 Principios de Cohesión (RCC): ¿Qué clases van juntas?

Estos principios ayudan a decidir qué clases deben agruparse en el mismo paquete:

9.1.1 REP (Equivalencia Reutilización/Revisión)

"La granularidad de la reutilización debe ser la granularidad de la revisión".

- Los paquetes son la unidad de distribución y, por tanto, la unidad de reutilización.
- Para ser reutilizable, un elemento debe estar gestionado por un sistema de distribución o control de versiones.

9.1.2 CRP (Reutilización Común)

"Todas las clases en un paquete deben ser reutilizadas juntas".

- Una dependencia de un paquete es una dependencia de todo lo que hay dentro.
- Si las clases no se usan siempre a la vez, deben separarse para evitar que el cliente tenga que verificar cambios en clases que no utiliza.
- Técnica: El uso del patrón Fachada facilita el cumplimiento de este principio.

9.1.3 CCP (Cierre Común)

"Las clases que cambian a la vez, deben estar juntas".

- Busca que los cambios se centren en un único paquete en lugar de dispersarse por todo el sistema.
- Es la versión del principio de Responsabilidad Única (SRP) aplicada a paquetes.

Existe cierta tensión entre principios, ya que REP y CRP benefician al reutilizador, mientras que CCP beneficia al equipo de mantenimiento. El diseño evoluciona de dominar el CCP (fases iniciales) a buscar maximizar REP/CRP (en fases estables).

9.2 Principios de Acoplamiento (ASS)

Estos principios gestionan las dependencias entre los paquetes ya formados:

9.2.1 ADP (Dependencias Acíclicas)

"Las dependencias entre paquetes no deben formar ciclos".

- Los ciclos crean dependencias transitivas donde un paquete acaba dependiendo de todos los demás, dificultando enormemente el desarrollo independiente.
- Solución: Romper los ciclos mediante el principio de Inversión de Dependencia (DIP) o creando un nuevo paquete intermedio.

9.2.2 SDP (Dependencias Estables)

"Un paquete debe depender solo de paquetes más estables que él".

- Estabilidad: Se define como la dificultad de cambiar un módulo.
- Un paquete es estable si es responsable (muchos dependen de él) e independiente (él no depende de nadie).

9.2.3 SAP (Abstracciones Estables)

"La abstracción de un paquete debe ser proporcional a su estabilidad".

- Los paquetes más estables (difíciles de cambiar) deben ser los más abstractos (interfaces/clases abstractas) para permitir su extensión sin modificación.
- Los paquetes inestables deben ser concretos.

10. Métricas de Diseño

El estado de un diseño de paquetes se puede medir cuantitativamente:

- Acoplamiento Aferente (Ca): Clases externas que dependen de mi paquete (Mide responsabilidad).
- Acoplamiento Eferente (Ce): Clases de mi paquete que dependen de fuera (Mide dependencia).
- Factor de Inestabilidad (I): $I = Ce / (Ca + Ce)$. Donde 0 es totalmente estable y 1 totalmente inestable.
- Abstractividad (A): Ratio de clases abstractas/interfaces sobre el total del paquete. Donde 0 es concreto y 1 abstracto.
- Distancia a la secuencia principal (D): $D = |A + I - 1|$.
 - **D = 0**: Equilibrio ideal entre estabilidad y abstracción (Secuencia Principal).
 - **D = 1**: Paquete totalmente desequilibrado.